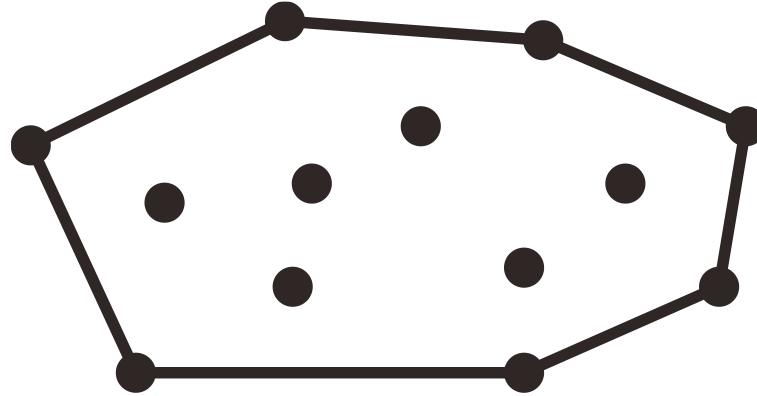


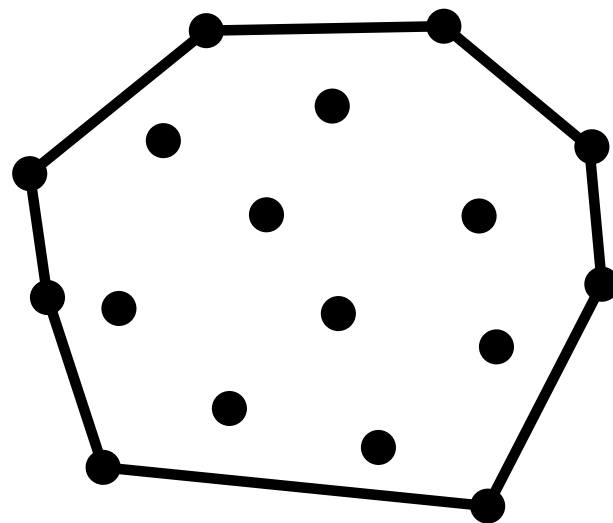
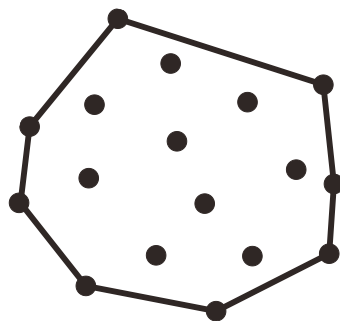


凸包 - Convex Hull

349班 14號 郭昀漢

大綱

- ▶ 凸包定義
- ▶ 二維凸包
- ▶ 應用範圍
- ▶ 演算法介紹
- ▶ 實作程式碼
- ▶ 執行結果
- ▶ 心得感想
- ▶ 資料來源



凸包定義

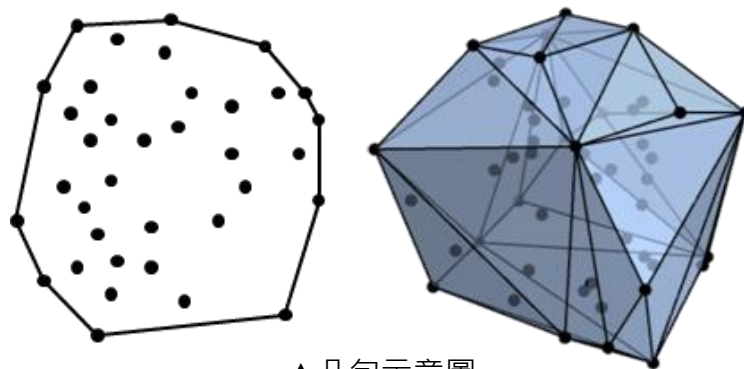
► 維基百科：

「在一個實數向量空間 V 中，對於給定集合 X ，所有包含 X 的凸集的交集 S 被稱為 X 的凸包。」

► 演算法筆記：

「在高維空間中有一群散佈各處的點，『凸包』是包覆這群點的所有外殼當中，表面積暨容積最小的一個外殼，而最小的外殼一定是凸的。」

「凸包是能包圍全部點的最小凸多邊形或凸殼。」

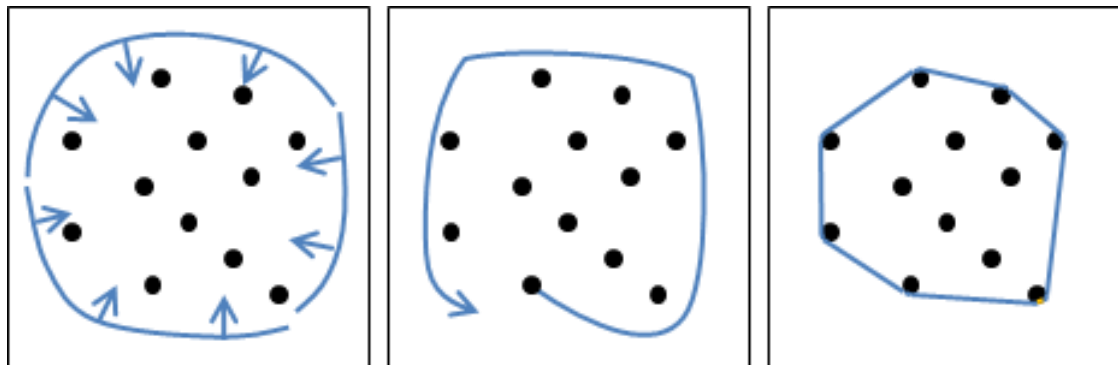


▲ 凸包示意圖

二維凸包

- ▶ 二維平面上的凸包是**凸多邊形**，
最外圍的點連線組成的多邊形即是凸包。
- ▶ 可將二維平面凸包視為一條剛好包著所有點的橡皮圈，
也可想像一條線繞最外圍的點。

▼二維平面凸包示意圖

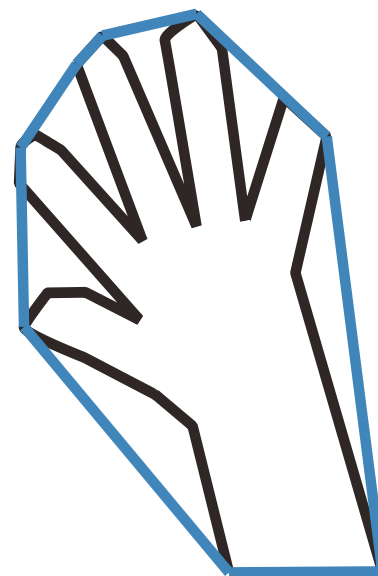
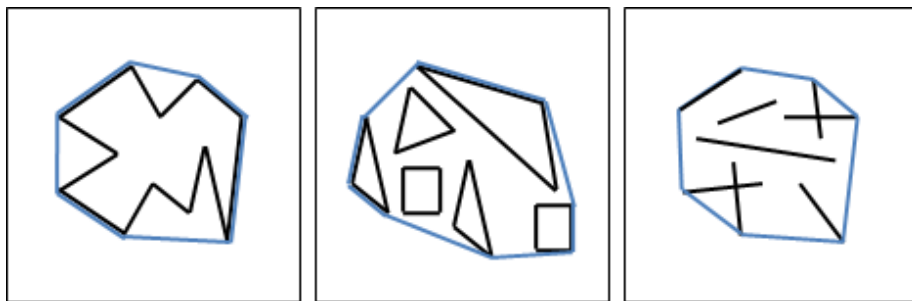


應用範圍

- ▶ 圖形處理
- ▶ 電腦視覺
- ▶ 地理資訊系統
- ▶ 無人地面車輛路徑規劃
- ▶ 二次表面巡航導彈任務規劃



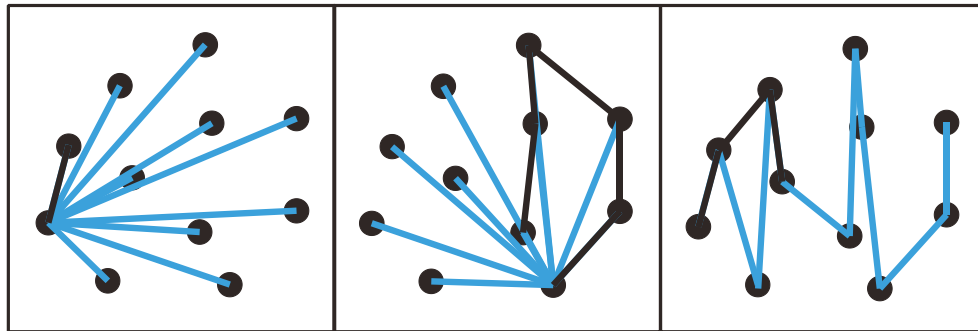
▼ 凸包可用於識別物體



演算法介紹

以下是三種凸包演算法解紹：

- ◆ 包裹法 (Gift Wrapping)
- ◆ 葛立恆掃描法 (Graham's Scan)
- ◆ 安德魯演算法 (Andrew's Algorithm)



演算法介紹

◆包裹法 (Gift Wrapping)

▶ 演算法說明：

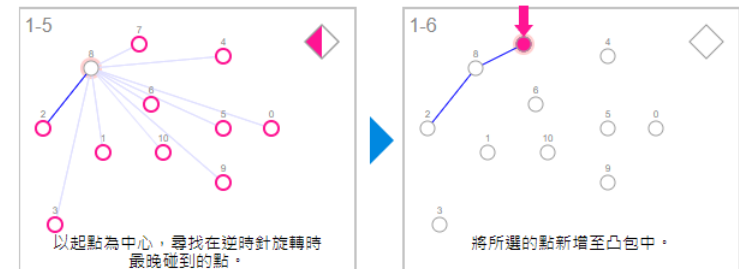
選定外圍一點作為起點，以起始點為中心尋找順或逆時針選轉時最晚碰到的點，
加入此點進入凸包並作為下一次旋轉的起點。
重複數次後，碰到原點即可得到凸包。

▶ 演算法動畫：

https://www.flag.com.tw/activity/F2708/exercise/1.0/algorithms/gift_wrapping/anim.html

▶ 時間複雜度

$O(\text{凸包邊數} * N)$



演算法介紹

◆ 葛立恆掃描法 (Graham's Scan)

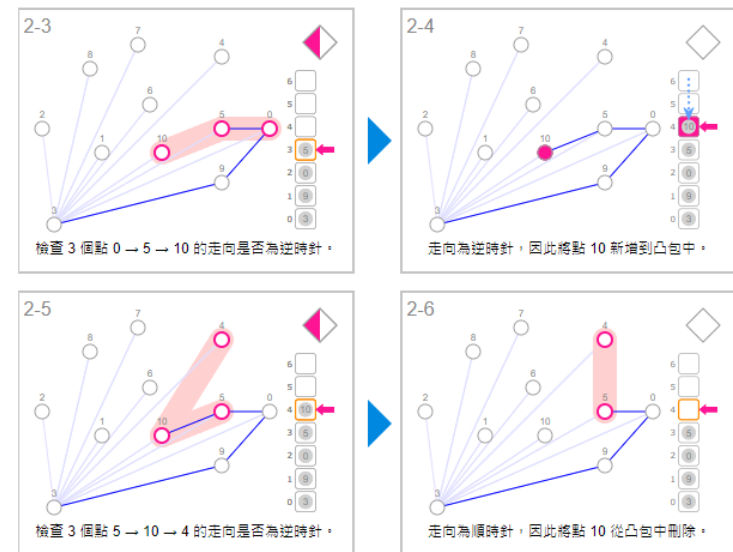
▶ 演算法說明：

選定外圍一點作為起點，以起始點為中心按角度排序，
依序加入點，並判斷倒數三個點的走向是順或逆時針，
若反方向就從凸包刪除。當所有點走訪完畢就得到凸包。

▶ 演算法動畫：

https://www.flag.com.tw/activity/F2708/exercise/1.0/algorithms/graham_scan/anim.html

▶ 時間複雜度 $O(N \log N)$



演算法介紹

◆安德魯演算法 (Andrew's Algorithm)

▶ 演算法說明：

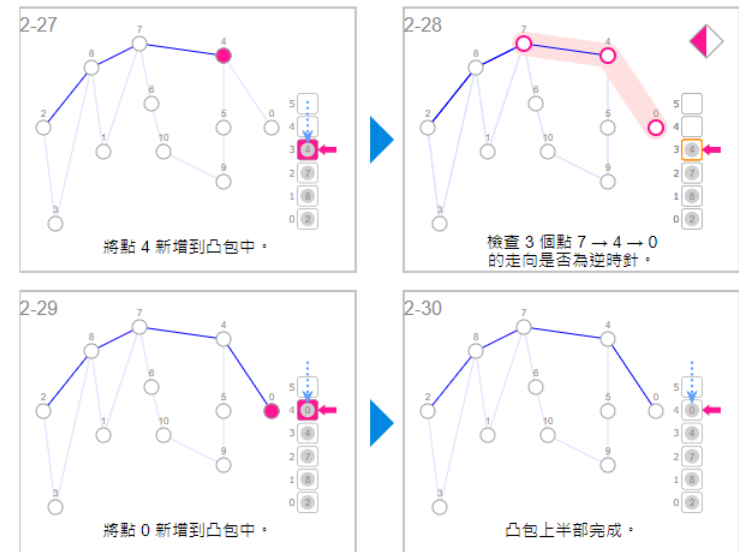
將所有點以 **X** 座標大小排序，當 **X** 座標相同則以 **Y** 座標大小排序。

與上個演算法做法類似，按排序掃描找到上半部凸包，再反向掃描找出下半部凸包，
上下兩半組合即是完整的凸包。

▶ 演算法動畫：

<https://www.flag.com.tw/activity/F2708/exercise/1.0/algorithms/andrew/anim.html>

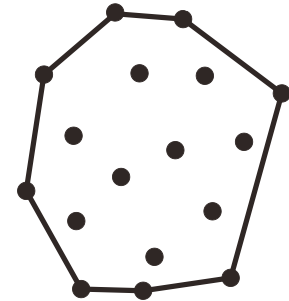
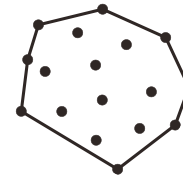
▶ 時間複雜度： $O(N \log N)$



實作程式碼

(1/6)

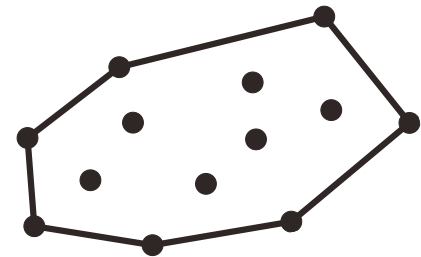
```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct point {int x, y;} p1, p2, p3;
5
6  vector <point> v1, v2, v4, v5, v6;
7  vector <pair<int, int>> v3;
8
9  // XY座標排序, X比Y先排
10 bool compare(point a, point b) {
11     return (a.y < b.y) || (a.y == b.y && a.x < b.x);
12 }
13
14 // 向量OA外積向量OB, 大於零表示從OA到OB為逆時針旋轉
15 int cross(point o, point a, point b) {
16     return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
17 }
18
19 // 兩點距離平方
20 int length(point a, point b) {
21     return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
22 }
23
24 // 以o點為中心點, a點較遠, b點較近
25 bool far(point o, point a, point b) {
```



實作程式碼

(2/6)

```
26 |     return length(o, a) > length(o, b);
27 | }
28 |
29 | // 角度相同時，距離長度的判斷
30 | bool compare_angle(point a, point b) {
31 |     int c = cross(v1[0], a, b);
32 |     return c > 0 || (c == 0 && length(v1[0], a) < length(v1[0], b));
33 | }
34 |
35 | // 包裹法
36 | void gw() {
37 |     int start = 0;
38 |     for (int i = 0; i < v1.size(); i++)
39 |         if (compare(v1[i], v1[start]))
40 |             start = i;
41 |     v2.push_back(v1[start]); // 記錄起點
42 |     int current = start;    // 當前的點
43 |     while (1) {
44 |         /* 尋找下一個要被包覆的點：
45 |            一、找出最外圍的一點
46 |            二、多點共線，則找最遠的一點 */
47 |         int next = current;
48 |         for (int i = 0; i < v1.size(); i++) {
49 |             int c = cross(v2.back(), v1[i], v1[next]);
50 |             if (c > 0 || c == 0 && far(v2.back(), v1[i], v1[next]))
```



實作程式碼

(3/6)

```
51         next = i;
52     }
53     if (next == start) break; // 回到起點結束
54     v2.push_back(v1[next]); // 記錄找到的點
55     current = next; // 前往下一個點
56 }
57 }
58
59 // 葛立恆掃描法
60 void gs() {
61     swap(v1[0], *min_element(v1.begin(), v1.end(), compare)); // 最低最左的點為起點
62     sort(v1.begin() + 1, v1.end(), compare_angle); // 其餘各點依角度排序
63     for (int i = 0; i < v1.size(); i++) {
64         if (i < 2) {
65             v6.push_back(v1[i]);
66             continue;
67         }
68         while (v6.size() >= 2 && cross(v6[v6.size() - 2], v6[v6.size() - 1], v1[i]) <= 0) {
69             v6.pop_back();
70         }
71         v6.push_back(v1[i]);
72     }
73 }
74
75 // 安德魯演算法
```

實作程式碼

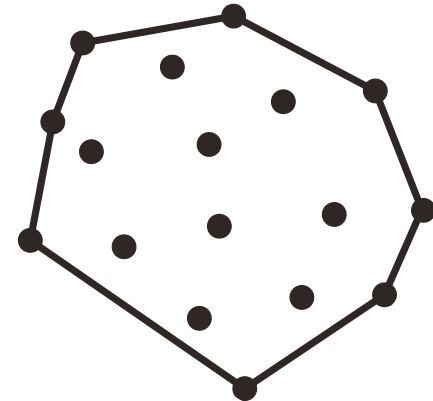
(4/6)

```
76 void aa() {
77     sort(v3.begin(), v3.end()); // 點的排序
78     for (int i = 0; i < v3.size(); i++) { // 包上半部
79         point p1 {.x = v3[i].first, .y = v3[i].second};
80         if (i < 2) {
81             v4.push_back(p1);
82             continue;
83         }
84         while (v4.size() >= 2) {
85             point p2 {.x = v4[v4.size() - 1].x, .y = v4[v4.size() - 1].y};
86             point p3 {.x = v4[v4.size() - 2].x, .y = v4[v4.size() - 2].y};
87             if (cross(p2, p3, p1) > 0) break;
88             v4.pop_back();
89         }
90         v4.push_back(p1);
91     }
92     for (int i = v3.size() - 1; i >= 0; i--) { // 包下半部
93         point p1 {.x = v3[i].first, .y = v3[i].second};
94         if (i > v3.size() - 2) {
95             v5.push_back(p1);
96             continue;
97         }
98         while (v5.size() >= 2) {
99             point p2 {.x = v5[v5.size() - 1].x, .y = v5[v5.size() - 1].y};
100            point p3 {.x = v5[v5.size() - 2].x, .y = v5[v5.size() - 2].y};
```

實作程式碼

(5/6)

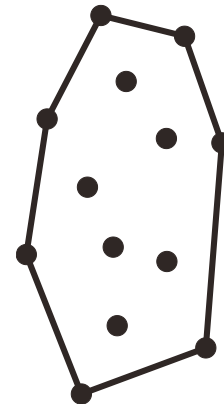
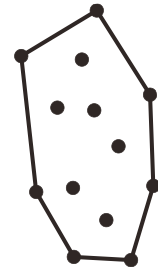
```
101         if (cross(p2, p3, p1) > 0) break;
102         v5.pop_back();
103     }
104     v5.push_back(p1);
105 }
106 }
107
108 int main() {
109     int n, x, y;    // 輸入點座標
110     cout << "< 請輸入點的總數 >\n點的個數:";
111     cin >> n;
112     if (n == 0) cout << "\\\\無點可包//";
113     cout << '\n';
114     for (int i = 0; i < n; i++) {
115         cout << "第 " << i + 1 << " 個座標:\n";
116         cout << "  x = ";
117         cin >> x;
118         cout << "  y = ";
119         cin >> y;
120         point p {.x = x, .y = y};
121         v1.push_back(p);
122         v3.push_back(make_pair(x, y));
123     }
124
125     gw();    // 包裹法
```



實作程式碼

(6/6)

```
126 gs(); // 葛立恆掃描法
127 aa(); // 安德魯演算法
128
129 cout << "\n< 執行結果 >\n"; // 輸出執行結果
130 cout << "使用「包裹法」得到的凸包:\n";
131 for (int i = 0; i < v2.size(); i++) {
132     cout << "(" << v2[i].x << ", " << v2[i].y << ")";
133     if (i < v2.size() - 1) cout << ",";
134 }
135 cout << "\n使用「葛立恆掃描法」得到的凸包:\n";
136 for (int i = 0; i < v6.size(); i++) {
137     cout << "(" << v6[i].x << ", " << v6[i].y << ")";
138     if (i < v6.size() - 1) cout << ",";
139 }
140 cout << "\n使用「安德魯演算法」得到的凸包:\n";
141 for (int i = 0; i < v4.size(); i++) {
142     cout << "(" << v4[i].x << ", " << v4[i].y << ")";
143     if (i < v4.size() - 1) cout << ",";
144 }
145 if (v5.size() > 2) cout << ",";
146 for (int i = 1; i < v5.size() - 1; i++) {
147     cout << "(" << v5[i].x << ", " << v5[i].y << ")";
148     if (i < v5.size() - 2) cout << ",";
149 }
150 }
```



執行結果

► 輸入：

一. 點的數量 (例：15)

二. 所有點座標

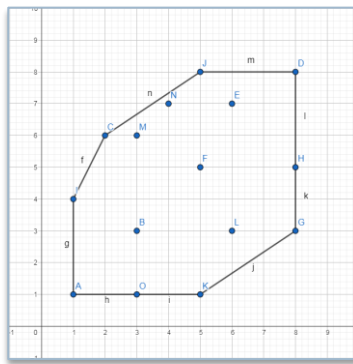
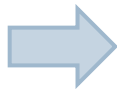
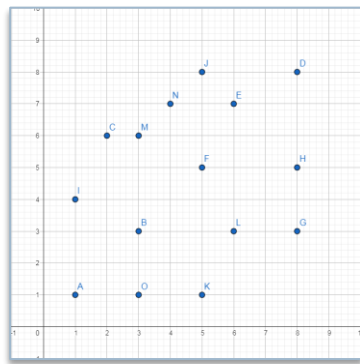
(例：1 1 1 4 2 6 3 1 3 3 3 6 4 7 5 1 5 5 5 8 6 3 6 7 8 3 8 5 8 8)

► 輸出：

一. 使用「包裹法」得到的凸包

二. 使用「葛立恆掃描法」得到的凸包

三. 使用「安德魯演算法」得到的凸包



```
< 請輸入點的總數 >
點的個數：15

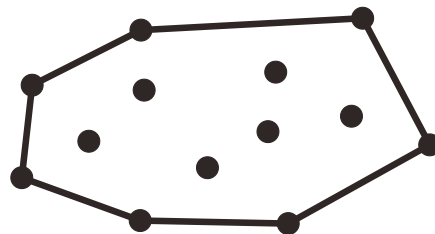
第 1 個座標：
x = 1
y = 1
第 2 個座標：
x = 1
y = 4
第 3 個座標：
x = 2
y = 6
第 4 個座標：
x = 3
y = 1
第 5 個座標：
x = 3
y = 3
第 6 個座標：
x = 3
y = 6
第 7 個座標：
x = 4
y = 7
第 8 個座標：
x = 5
y = 1
第 9 個座標：
x = 5
y = 5
第 10 個座標：
x = 5
y = 8
第 11 個座標：
x = 6
y = 3
第 12 個座標：
x = 6
y = 7
第 13 個座標：
x = 8
y = 3
第 14 個座標：
x = 8
y = 5
第 15 個座標：
x = 8
y = 8

< 執行結果 >
使用「包裹法」得到的凸包：
<1, 1>,<5, 1>,<8, 3>,<8, 8>,<5, 8>,<2, 6>,<1, 4>
使用「葛立恆掃描法」得到的凸包：
<1, 1>,<5, 1>,<8, 3>,<8, 8>,<5, 8>,<2, 6>,<1, 4>
使用「安德魯演算法」得到的凸包：
<1, 1>,<1, 4>,<2, 6>,<5, 8>,<8, 8>,<8, 3>,<5, 1>

Process exited after 59.49 seconds with return value 0
請按任意鍵繼續 . . .
```


心得感想

- ▶ 我想做**凸包**這個主題是因為我剛好在書上翻到，發現這些演算法概念、執行步驟看起來滿酷的，想嘗試自己寫寫看程式。實際實做過後，我發現並不如想像中的簡單，過程中我因為弄錯點在容器的位置、搞錯優先順序、拿錯容器、用錯pair與struct，犯了許多錯誤，但也是這些錯誤，讓我在每次修正都獲得不少成長，無形中強化我上網找資料的能力，也使得我更進一步的熟悉程式與演算法。
- ▶ 這次實作成功教會我許多我以前未曾接觸過的知識，也帶給我無比的成就感，期待未來我能嘗試更多不同的演算法，探索不同的領域。



資料來源

- ▶ 凸包 - 維基百科

<https://zh.wikipedia.org/zh-tw/%E5%87%B8%E5%8C%85>

- ▶ Convex Hull - 演算法筆記

<https://web.ntnu.edu.tw/~algo/ConvexHull.html>

- ▶ 基於網格架構之凸包快速演算法

<https://www.airitilibrary.com/Article/Detail/P20191126001-201911-201911260014-201911260014-150-155>

- ▶ 會動的演算法

<https://www.flag.com.tw/activity/F2708/exercise/books/>





謝謝大家

